



UNIVERSITÀ DEGLI STUDI DI MESSINA

DIPARTIMENTO DI SCIENZE MATEMATICHE E INFORMATICHE,
SCIENZE FISICHE E SCIENZE DELLA TERRA

Laurea Triennale in Informatica L31
Curriculum: Tecnologie Informatiche
Tirocinio

TITOLO TESI

Tutor dell'ente ospitante:

Prof. Antonio Celesti

Candidato:

Sandi Russo

Tutor accademico:

Prof. Lorenzo Carnevale

Anno Accademico:

2025/2026

Indice

1	Introduzione	2
1.1	Contestualizzazione	2
1.2	Definizione del problema	3
1.3	Obiettivi del lavoro svolto	4
2	Stato dell'arte	5
2.1	Il modello Property Graph	5
2.2	Neo4j Enterprise e GDS	7
2.3	Graph Embeddings e Graph Neural Networks	8
2.4	GraphSAGE	9
2.4.1	Meccanismo di funzionamento	10
2.4.2	Pipeline per la link prediction	10
2.4.3	Vantaggi e limitazioni	11
2.5	TransE	12
2.5.1	Meccanismo di funzionamento	13
2.5.2	Vantaggi e limitazioni	14
2.6	HashGNN	14
2.6.1	Meccanismo di funzionamento	14
2.6.2	Vantaggi e limitazioni	15
3	Materiali e metodi	16
3.1	Il dataset: PharMeBiNet V3	16

3.1.1	Analisi esplorativa del dataset	16
3.2	Infrastruttura del cluster Neo4j	18
3.2.1	Architettura generale	18
3.2.2	Configurazione della memoria	19
3.2.3	Configurazione di rete e WSL2	19
3.2.4	Automazione con Ansible	19
3.3	Caricamento del dataset	20
3.4	Pipeline di estrazione dati	21
3.5	Implementazione dei modelli	22
3.5.1	GraphSAGE	22
3.5.2	TransE	22
3.5.3	HashGNN	22
3.5.4	Training loop	23
3.6	Protocollo di valutazione	24
3.6.1	Metrica principale: AUC-ROC	24
3.6.2	Benchmark di scalabilità	24
3.6.3	Pipeline automatizzata	24
4	Risultati e Discussione	26
4.1	Benchmark infrastrutturale	26
4.1.1	Tempi di estrazione al variare dei nodi attivi	26
4.1.2	Analisi dei risultati infrastrutturali	27
4.2	Risultati della Link Prediction	28
4.2.1	Metriche per modello	28
4.2.2	Confronto grafico	28
4.3	Discussione	28
4.3.1	GraphSAGE: precisione massima, convergenza rapida	28
4.3.2	HashGNN: robusto ma lento	31
4.3.3	TransE: il vincitore dell'efficienza	31

4.3.4	Il trade-off accuratezza-efficienza	32
4.3.5	Confronto con l'ipotesi iniziale	32
5	Conclusioni	34
5.1	Sintesi del lavoro	34
5.2	Risposte alle domande di ricerca	35
5.2.1	Quale algoritmo per la link prediction su grafi biomedici eterogenei?	35
5.2.2	Come scala il cluster Neo4j su pipeline di estrazione batch? . . .	35
5.3	Contributi	36
5.4	Limitazioni	36
5.5	Sviluppi futuri	37

Elenco delle figure

2.1	Esempio di Property Graph con nodi etichettati e proprietà	6
2.2	Meccanismo di funzionamento della libreria GDS in Neo4j.	8
2.3	Architettura end-to-end: dal Knowledge Graph eterogeneo (nodi Chemical e Disease) all'estrazione degli embedding nello spazio latente, fino al task di Link Prediction tramite rete neurale (MLP).	8
2.4	Il meccanismo di aggregazione di GraphSAGE: i vicini campionati vengono aggregati e combinati con la rappresentazione del nodo target.	11
2.5	Pipeline completa di GraphSAGE per la link prediction Chemical→Disease.	12
2.6	L'intuizione geometrica di TransE: la relazione $\mathbf{r}$ è un vettore di traslazione da $\mathbf{h}$ a $\mathbf{t}$ nello spazio degli embedding.	13
2.7	Architettura di HashGNN per la link prediction: Chemical e Disease mantengono spazi di embedding separati fino al decoder.	15
4.1	Confronto dei tempi di estrazione nelle tre configurazioni del cluster.	27
4.2	AUC-ROC medio con deviazione standard per i tre modelli su 5 run indipendenti.	29
4.3	Tempo medio di training per i tre modelli su 5 run indipendenti.	30
4.4	Trade-off tra accuratezza (AUC-ROC) ed efficienza computazionale (tempo di training) per i tre modelli.	32

Elenco delle tabelle

2.1	Confronto dei tre algoritmi analizzati.	9
3.1	Grado medio e massimo dei principali tipi di nodo in PharMeBiNet V3. .	18
3.2	Configurazione del cluster Neo4j Enterprise.	18
3.3	Iperparametri condivisi tra i tre modelli.	23
4.1	Tempi di estrazione dati da Neo4j verso PyTorch Geometric al variare della configurazione del cluster.	26
4.2	Risultati della link prediction Chemical→Disease su 5 run indipendenti, 200 epoche.	28
4.3	Risultati dettagliati per singola run.	29

Listings

Sommario

L'efficacia e la scalabilità di tre algoritmi di Graph Machine Learning vengono analizzate applicandoli a un knowledge graph biomedico di grandi dimensioni. È stato scelto il dataset PharMeBiNet V3, una rete eterogenea con circa 8 milioni di nodi distribuiti su oltre 80 tipi e 53 milioni di relazioni distribuite a 410 tipi distinti, che interconnette composti chimici, geni, proteine e patologie provenienti da oltre 40 banche dati farmacologiche e biologiche.

Il problema studiato è la link prediction tra nodi *Chemical* e *Disease*, biologicamente rilevante nel contesto del drug repurposing, ovvero prevedere relazioni terapeutiche non ancora documentate tra farmaci esistenti e malattie note. Il sottografo estratto per gli esperimenti comprende 594.343 nodi Chemical, 23.580 nodi Disease e 123.361 relazioni dirette tra loro. I tre algoritmi confrontati sono GraphSAGE, TransE e HashGNN, scelti per le loro diverse strategie di rappresentazione, rispettivamente: omogenea con aggregazione dei vicini, traslazionale nello spazio vettoriale, ed eterogenea con proiezioni differenziate per tipo di nodo.

L'infrastruttura sperimentale è un cluster Neo4j Enterprise 2026.04.0 distribuito su quattro macchine virtuali cloud, nella configurazione di 1 primary e 3 secondary. Il benchmark di scalabilità valuta l'impatto del numero di nodi attivi sui tempi di estrazione dei dati verso la pipeline PyTorch Geometric, variando la configurazione tra 1, 2 e 4 nodi.

I risultati mostrano che GraphSAGE raggiunge la precisione più alta con un AUC-ROC medio di 0.993 su cinque run indipendenti, mentre TransE si rivela il modello più efficiente, registrando tempi di training circa tre volte inferiori pur mantenendo una qualità predittiva eccellente ($\text{AUC-ROC} > 0.98$). Sul fronte infrastrutturale, il cluster distribuito non produce speedup significativi su query sequenziali di estrazione batch, evidenziando come il collo di bottiglia sia legato all'overhead di coordinamento di rete piuttosto che alla mera disponibilità di risorse computazionali.

Acknowledgement

Capitolo 1

Introduzione

1.1 Contestualizzazione

Negli ultimi decenni la quantità di dati prodotti e consumati a livello globale è cresciuta a ritmi senza precedenti. Secondo le stime di Statista, nel 2024 il volume globale delle informazioni create e consumate supererà i 149 zettabyte (10^{21} byte) [1], confermando una crescita esponenziale del volume dei dati prodotti a livello mondiale.

Diversi fattori strutturali spiegano questa esplosione: la diffusione dell'Internet of Things (IoT), la trasformazione digitale della sanità e l'emergere delle tecnologie di *Next-Generation Sequencing* (NGS). Il dato non rappresenta più soltanto un sottoprodotto dell'attività digitale, ma una risorsa strategica che richiede nuovi strumenti di gestione e analisi. Si parla del fenomeno dei **Big Data**, descritto attraverso il concetto delle "cinque V": il *volume* massiccio dei dataset, la *velocità* con cui i flussi vengono generati e analizzati, la *varietà* delle informazioni, la *veridicità* della fonte e il *valore* dell'analisi [2]. A questi si aggiunge spesso anche la *variabilità*, che riflette l'eterogeneità e l'instabilità dei dati nel tempo e nei diversi domini applicativi. In bioinformatica, questo impatto è ancora più evidente. La ricerca farmacologica e lo studio delle reti metaboliche richiedono l'integrazione di dati molto vari, dove il legame tra le entità biologiche assume un'importanza informatica almeno pari a quella delle entità stesse.

1.2 Definizione del problema

Per decenni, la gestione dei dati è stata dominata dai sistemi **RDBMS** (*Relational Database Management System*), basati su un modello rigorosamente strutturato e su una teoria consolidata che garantisce consistenza e integrità delle informazioni. Tuttavia, mostra limiti evidenti quando viene applicato a domini caratterizzati da forte interconnessione, come le reti biologiche e biomediche.

In tali scenari, le entità non possono essere considerate in modo isolato, ma devono essere interpretate all'interno di una fitta rete di relazioni. Quando una query richiede l'attraversamento di molteplici livelli di connessione, il modello relazionale è costretto a ricorrere a un numero elevato di operazioni di *JOIN*. Questo comportamento introduce un overhead computazionale significativo, che cresce rapidamente con la profondità della query e con la densità della rete, compromettendo la scalabilità delle interrogazioni.

Per far fronte a queste limitazioni, si è progressivamente affermato il paradigma **NoSQL** (*"Not Only SQL"*) [3], termine coniato da Carlo Strozzi nel 1998 per indicare una famiglia eterogenea di sistemi di gestione dei dati caratterizzati da flessibilità dello schema (*schema-agnostic*), scalabilità orizzontale e capacità di distribuzione su infrastrutture commodity. Tale paradigma non sostituisce il modello relazionale, ma lo affianca in contesti in cui la rigidità strutturale rappresenta un vincolo.

All'interno dell'ecosistema NoSQL, i database a grafo hanno assunto un ruolo centrale nella modellazione di sistemi complessi. Essi consentono di rappresentare in modo naturale entità e relazioni, preservando la struttura intrinseca del dominio: i nodi modellano le entità biologiche, mentre gli archi rappresentano le interazioni tra esse. Questa rappresentazione si adatta particolarmente bene alla bioinformatica, dove il valore informativo risiede spesso nelle connessioni piuttosto che nei singoli elementi.

Neo4j rappresenta oggi uno dei database a grafo nativi più diffusi e maturi. Attraverso la libreria *Graph Data Science* (GDS), il sistema mette a disposizione algoritmi scalabili per l'analisi dei grafi e per l'apprendimento automatico su strutture complesse, integrando in un unico ecosistema storage, query engine e pipeline di analisi.

1.3 Obiettivi del lavoro svolto

Il presente lavoro propone un’analisi congiunta teorica e sperimentale basata sul knowledge graph biomedico PharMeBiNet V3, una rete eterogenea di grandi dimensioni composta da circa 8 milioni di nodi e 53 milioni di relazioni, provenienti da molteplici database farmacologici e biologici. La complessità strutturale del grafo lo rende un banco di prova particolarmente rappresentativo per lo studio di algoritmi di Graph Machine Learning in contesti reali.

Il task considerato è la *link prediction* tra nodi di tipo *Chemical* e *Disease*, problematica centrale nel dominio del *drug repurposing*. L’obiettivo consiste nell’identificare potenziali relazioni terapeutiche non ancora documentate, sfruttando la struttura del grafo e le informazioni latenti codificate nelle sue connessioni.

L’analisi si concentra sul confronto tra tre algoritmi: GraphSAGE, TransE e HashGNN. Tali modelli sono stati selezionati in quanto rappresentano tre paradigmi distinti di apprendimento su grafi: l’aggregazione di informazioni nei vicini (GraphSAGE), la modellazione geometrica delle relazioni nello spazio latente (TransE) e la gestione esplicita dell’eterogeneità tramite rappresentazioni differenziate per tipo di nodo (HashGNN). Questa diversità metodologica consente di valutare in modo più completo l’impatto delle scelte architetturali sul problema considerato.

Dal punto di vista infrastrutturale, il lavoro prevede la progettazione e l’implementazione di un cluster Neo4j Enterprise distribuito su quattro macchine virtuali in ambiente Azure, configurato secondo un’architettura primary-secondary. L’obiettivo è analizzare l’impatto della scalabilità orizzontale sui tempi di estrazione dei dati necessari alla pipeline di training basata su PyTorch Geometric, variando il numero di nodi attivi nel cluster.

In questo contesto, la valutazione non si limita alla sola accuratezza predittiva dei modelli, ma estende l’analisi alle prestazioni end-to-end della pipeline, con l’obiettivo di quantificare il reale contributo dell’infrastruttura distribuita in scenari di data engineering su grafi biomedici di grande scala.

Capitolo 2

Stato dell'arte

2.1 Il modello Property Graph

Il **Property Graph**, noto anche come *Labeled Property Graph* (LPG), è un modello di dati emerso nei primi anni 2000 per rappresentare reti di entità interconnesse in modo naturale ed efficiente. Un property graph è un multigrafo orientato in cui i vertici rappresentano entità del dominio applicativo e gli archi rappresentano le relazioni tra esse. A differenza del modello relazionale, in cui le associazioni tra entità sono codificate implicitamente tramite chiavi esterne, nel modello a grafo le relazioni esistono come strutture autonome, tipate e arricchibili con proprietà, rendendo la loro navigazione un'operazione nativa e non derivata.

Il modello si compone strutturalmente di due elementi fondamentali:

- I **nodi** (o vertici): rappresentano le entità. Sono caratterizzati da un identificatore univoco, un insieme di *label* che ne classificano la natura semantica e una collezione di proprietà espresse come coppie chiave-valore. Le label consentono di raggruppare i nodi in insiemi omogenei e possono essere modificate dinamicamente senza alterare lo schema del grafo. La possibilità di assegnare zero o più label simultaneamente permette di modellare entità appartenenti a categorie multiple, senza ricorrere a tabelle di associazione aggiuntive.

- Gli **archi** (o relazioni): descrivono le connessioni tra i nodi. Ogni arco possiede un identificatore univoco, un nodo sorgente e un nodo destinazione, un’etichetta che ne definisce il tipo semantico e un’eventuale collezione di proprietà. A differenza dei nodi, ogni arco deve avere un tipo e una direzione esplicita, sebbene quest’ultima possa essere ignorata dal motore di calcolo durante le query che richiedono un attraversamento non orientato.

Questa flessibilità strutturale si adatta perfettamente alla modellazione di domini complessi. Nel contesto biomedico, ad esempio, consente di rappresentare in modo diretto l’interazione tra un composto chimico (nodo con label *Chemical*) e una patologia (nodo con label *Disease*) attraverso un arco tipato (es. *relates_to*), potendo inoltre memorizzare all’interno dell’arco stesso le proprietà specifiche di quella interazione, come il peso statistico o la fonte bibliografica della scoperta.

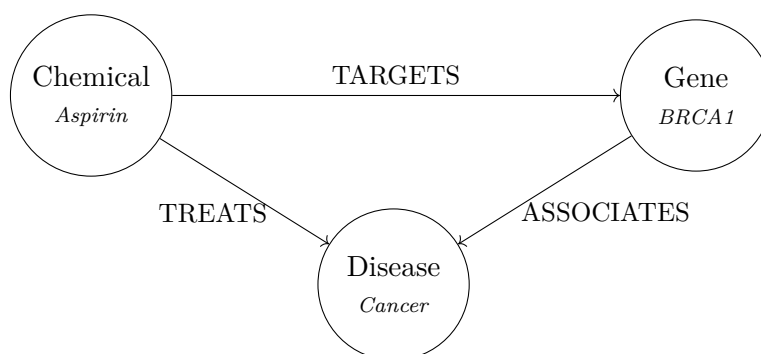


Figura 2.1: Esempio di Property Graph con nodi etichettati e proprietà

Il vantaggio computazionale più rilevante del modello LPG rispetto ai sistemi relazionali emerge nelle query di attraversamento (*graph traversal*). In un RDBMS, la navigazione tra entità correlate richiede operazioni di *JOIN* la cui complessità cresce in modo superlineare con la profondità della query e la cardinalità delle tabelle coinvolte. In un database a grafo nativo, ogni passo di traversata ha un costo costante: dato un nodo, l’accesso ai suoi vicini diretti avviene tramite puntatori fisici senza consultare strutture di indice globali, rendendo il modello LPG particolarmente adatto alla bioinformatica.

2.2 Neo4j Enterprise e GDS

Neo4j è un database a grafo **nativo**, il che significa che implementa il modello a grafo fino al livello di memorizzazione fisica. A differenza dei sistemi che emulano un grafo sopra una tecnologia preesistente, i dati in Neo4j vengono memorizzati esattamente nel modo in cui vengono modellati, senza strati di astrazione intermedi.

Il meccanismo architetturale che rende possibile questa efficienza è l'**index-free adjacency**: ogni nodo memorizza fisicamente i puntatori diretti agli archi incidenti, i quali a loro volta contengono riferimenti immediati ai nodi sorgente e destinazione. Attraversare una relazione non richiede quindi la consultazione di strutture di indice globali, rendendo il costo di ogni singolo passo di traversata $O(1)$ rispetto alla dimensione complessiva del grafo.

Il linguaggio di interrogazione nativo di Neo4j è **Cypher**, un linguaggio dichiarativo ottimizzato per i grafi, concettualmente simile a SQL ma progettato per esprimere pattern strutturali e query di attraversamento in modo intuitivo. Cypher è anche alla base del progetto *openCypher* e ha contribuito alla definizione dello standard **GQL** (*Graph Query Language*).

Per l'analisi algoritmica su larga scala, Neo4j mette a disposizione la libreria **Graph Data Science** (GDS), un'estensione che fornisce implementazioni parallele di algoritmi su grafi esposti come procedure Cypher. La libreria introduce il concetto di **graph projection**: prima dell'esecuzione di un algoritmo, il sottografo di interesse viene proiettato in una struttura in memoria separata, denominata *graph catalog*, ottimizzata per il calcolo analitico. Questo meccanismo disaccoppia il layer transazionale da quello analitico, consentendo l'esecuzione di algoritmi massivamente paralleli senza interferire con le operazioni correnti sul database.

In questa tesi, Neo4j Enterprise 2026.04.0 viene utilizzato come layer di storage distribuito per PharMeBiNet V3. Il cluster è composto da quattro nodi, un primary e tre secondary, e la libreria GDS costituisce la base concettuale del meccanismo di estrazione del sottografo verso PyTorch Geometric, descritto nel capitolo dei metodi.

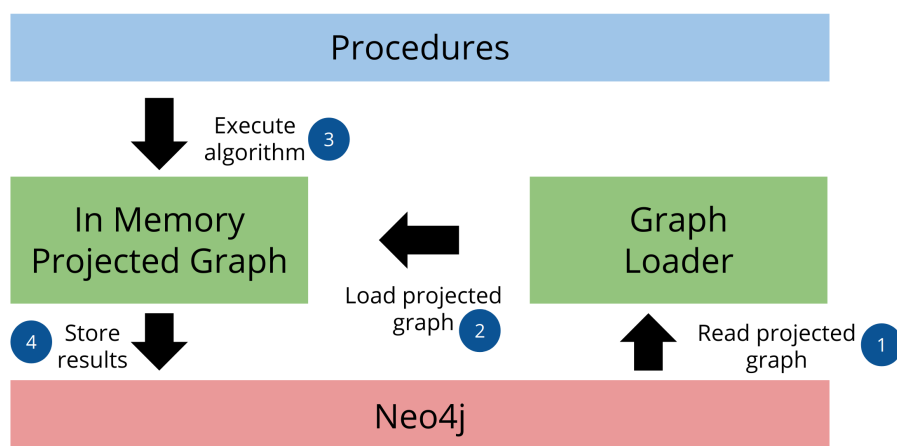


Figura 2.2: Meccanismo di funzionamento della libreria GDS in Neo4j.

2.3 Graph Embeddings e Graph Neural Networks

I **graph embedding** sono funzioni che proiettano i nodi di un grafo in uno spazio vettoriale continuo a bassa dimensionalità, preservando proprietà strutturali e semantiche della rete originale. L'obiettivo è trasformare entità discrete — nodi identificati da un ID o da una stringa — in vettori numerici su cui applicare algoritmi di Machine Learning standard.

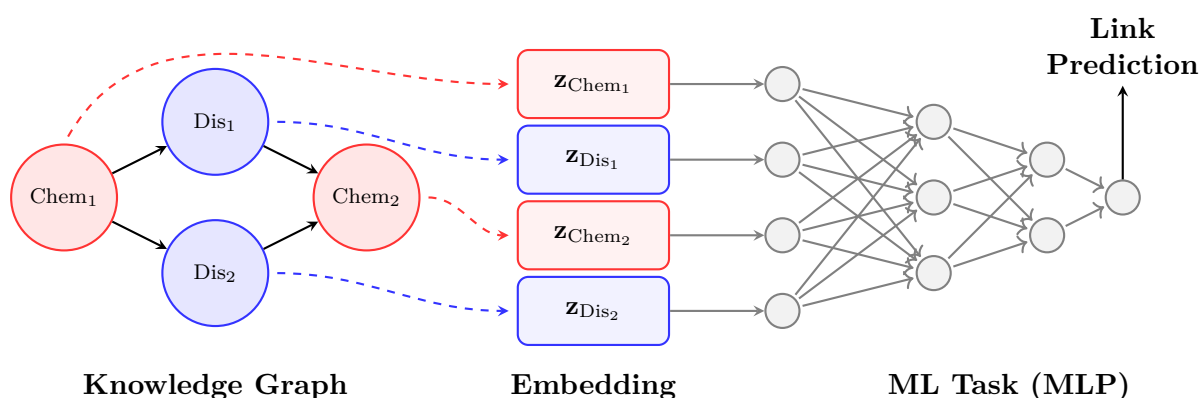


Figura 2.3: Architettura end-to-end: dal Knowledge Graph eterogeneo (nodi Chemical e Disease) all'estrazione degli embedding nello spazio latente, fino al task di Link Prediction tramite rete neurale (MLP).

Le **Graph Neural Networks** (GNN) sono la famiglia di modelli che apprende questi embedding direttamente dalla struttura del grafo, senza richiedere feature ingegnerizzate manualmente. Il meccanismo comune a tutte le GNN è il **message passing**: per ogni nodo, l'embedding viene aggiornato iterativamente aggregando le rappresentazioni dei nodi vicini. Dopo k iterazioni, l'embedding di un nodo cattura informazioni strutturali fino a k salti di distanza nella rete.

Formalmente, dato un nodo v al layer l , l'aggiornamento segue lo schema:

$$\mathbf{h}_v^{(l)} = \text{UPDATE}(\mathbf{h}_v^{(l-1)}, \text{AGGREGATE}(\{\mathbf{h}_u^{(l-1)} : u \in \mathcal{N}(v)\})) \quad (2.1)$$

dove $\mathcal{N}(v)$ indica l'insieme dei vicini di v , e le funzioni AGGREGATE e UPDATE variano a seconda dell'architettura specifica.

I tre algoritmi analizzati in questa tesi rappresentano tre approcci distinti a questo problema, differenziandosi per strategia di aggregazione, gestione dell'eterogeneità del grafo e tipo di task per cui sono stati progettati. La tabella 2.1 ne riassume le caratteristiche principali.

Algoritmo	Approccio	Eterogeneità	Task ideale
GraphSAGE	Aggregazione vicini	Omogeneo	Link prediction, node classification
TransE	Traslazione vettoriale	Relazioni tipizzate	Link prediction su KG
HashGNN	Hashing + proiezione	Eterogeneo nativo	Grafi multi-tipo

Tabella 2.1: Confronto dei tre algoritmi analizzati.

2.4 GraphSAGE

GraphSAGE (*Graph SAmple and aggreGatE*) è un algoritmo di graph embedding induttivo proposto da Hamilton et al. nel 2017. A differenza degli approcci trasduttivi, che calcolano embedding solo per i nodi presenti durante il training, GraphSAGE apprende una **funzione di aggregazione** generalizzabile a nodi non visti, rendendolo adatto a grafi dinamici e a larga scala.

2.4.1 Meccanismo di funzionamento

L'intuizione alla base di GraphSAGE è semplice: l'identità di un nodo è definita dal suo vicinato. Invece di apprendere un vettore per ogni nodo individualmente, l'algoritmo apprende come aggregare le feature dei vicini per produrre una rappresentazione del nodo stesso.

Il forward pass si articola in due fasi per ogni layer l :

1. **Campionamento:** per ogni nodo v , viene selezionato un sottoinsieme fisso $\mathcal{S}(v) \subseteq \mathcal{N}(v)$ di vicini. Il campionamento è necessario per controllare il costo computazionale su nodi ad alto grado.
2. **Aggregazione:** le rappresentazioni dei vicini campionati vengono aggregate e combinate con la rappresentazione corrente del nodo.

Formalmente:

$$\mathbf{h}_{\mathcal{N}(v)}^{(l)} = \text{AGGREGATE}_l(\{\mathbf{h}_u^{(l-1)} : u \in \mathcal{S}(v)\}) \quad (2.2)$$

$$\mathbf{h}_v^{(l)} = \sigma\left(\mathbf{W}^{(l)} \cdot \left[\mathbf{h}_v^{(l-1)} \parallel \mathbf{h}_{\mathcal{N}(v)}^{(l)}\right]\right) \quad (2.3)$$

dove $\parallel$ indica la concatenazione, $\mathbf{W}^{(l)}$ è la matrice di pesi apprendibile al layer l e σ è una funzione di attivazione non lineare. Gli aggregatori supportati includono *Mean* (media delle rappresentazioni dei vicini), *Pool* (max-pooling dopo una trasformazione lineare) e *LSTM* (aggregazione sequenziale su una permutazione casuale dei vicini).

2.4.2 Pipeline per la link prediction

Nel contesto di questa tesi, GraphSAGE viene impiegato per il task di link prediction tra nodi *Chemical* e *Disease*. La pipeline si articola in tre componenti distinti.

Encoder: poiché i nodi Chemical e Disease in PharMeBiNet non possiedono feature numeriche — sono identificati solo da stringhe come `DB00620` o `MONDO:0000004` — le feature iniziali sono embedding learnable, una tabella di vettori inizializzati casualmente

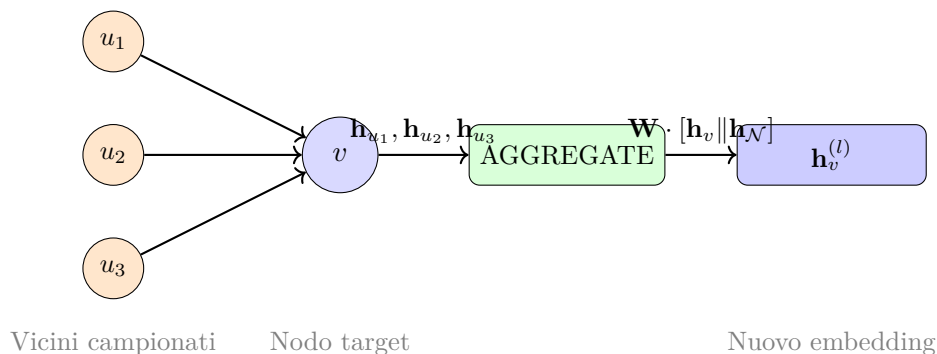


Figura 2.4: Il meccanismo di aggregazione di GraphSAGE: i vicini campionati vengono aggregati e combinati con la rappresentazione del nodo target.

e ottimizzati durante il training. Gli embedding di Chemical e Disease vengono fusi in un unico tensore omogeneo tramite concatenazione, e gli indici Disease vengono traslati di `num_chemical` posizioni per evitare collisioni. Due layer SAGEConv propagano l'informazione strutturale del vicinato.

Decoder: dati gli embedding $\mathbf{z}_{chem}$ e $\mathbf{z}_{dis}$ di una coppia di nodi, il decoder li concatena e li passa attraverso un MLP (*Multi-Layer Perceptron*) con attivazione finale sigmoid, producendo uno score $p \in [0, 1]$ che rappresenta la probabilità che il link esista.

Training: il modello viene addestrato su link positivi (archi reali) e link negativi generati casualmente, minimizzando la *binary cross-entropy loss*. Il 10% degli archi viene tenuto nascosto per la valutazione finale tramite `RandomLinkSplit`.

2.4.3 Vantaggi e limitazioni

GraphSAGE è induttivo, scalabile e converge rapidamente. Il suo principale limite su PharMeBiNet è che tratta il grafo come omogeneo: non distingue i 410 tipi di relazione presenti nel dataset, aggregando indiscriminatamente vicini di tipo diverso. Su nodi ad alto grado come le proteine (grado medio 232), il campionamento fisso introduce rumore informativo perdendo parte del contesto biologico.

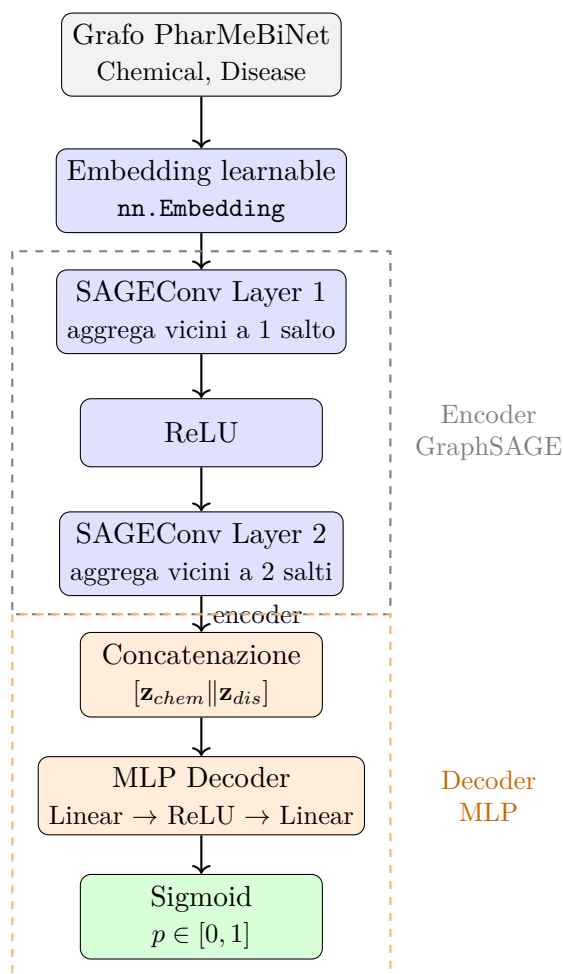


Figura 2.5: Pipeline completa di GraphSAGE per la link prediction Chemical→Disease.

2.5 TransE

TransE (*Translating Embeddings*) è un modello di knowledge graph embedding proposto da Bordes et al. nel 2013. A differenza delle GNN che aggregano informazioni strutturali dai vicini, TransE apprende embedding per entità e relazioni modellando le relazioni come **traslazioni nello spazio vettoriale**.

2.5.1 Meccanismo di funzionamento

L'intuizione di TransE è geometrica: data una tripla valida (h, r, t) — dove h è il nodo testa, r la relazione e t il nodo coda — il modello ottimizza gli embedding affinché $\mathbf{h} + \mathbf{r} \approx \mathbf{t}$.

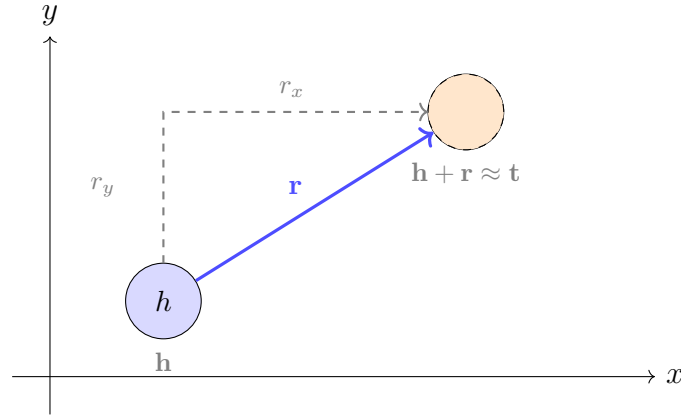


Figura 2.6: L'intuizione geometrica di TransE: la relazione $\mathbf{r}$ è un vettore di traslazione da $\mathbf{h}$ a $\mathbf{t}$ nello spazio degli embedding.

La funzione di scoring per una tripla (h, r, t) è la distanza L2 tra $\mathbf{h} + \mathbf{r}$ e $\mathbf{t}$:

$$f(h, r, t) = \|\mathbf{h} + \mathbf{r} - \mathbf{t}\|_2 \quad (2.4)$$

Un valore basso indica una tripla plausibile, un valore alto indica una tripla falsa. Il training usa una **margin ranking loss**:

$$\mathcal{L} = \sum_{(h,r,t) \in S} \sum_{(h',r,t') \in S'} \max(0, \gamma + f(h, r, t) - f(h', r, t')) \quad (2.5)$$

dove S è l'insieme delle triple positive, S' quello delle triple negative generate per corruzione, e γ è il margine. La loss penalizza i casi in cui il punteggio della tripla negativa non supera quello della tripla positiva di almeno γ .

2.5.2 Vantaggi e limitazioni

TransE è computazionalmente efficiente e cattura esplicitamente la semantica delle relazioni tramite il vettore $\mathbf{r}$, il che lo rende particolarmente adatto a knowledge graph con molti tipi di relazione come PharMeBiNet. Il suo principale limite è che è un modello **trasduttivo**: non generalizza a nodi non visti durante il training. Presenta inoltre difficoltà nel modellare relazioni simmetriche ($h \rightarrow t$ e $t \rightarrow h$ dello stesso tipo) poiché la traslazione è direzionale per definizione.

2.6 HashGNN

HashGNN è un algoritmo di graph embedding progettato nativamente per grafi eterogenei, proposto da Ni et al e successivamente integrato nella libreria Neo4j GDS. A differenza di GraphSAGE, che fonde i diversi tipi di nodo in un unico spazio omogeneo, HashGNN mantiene **spazi di rappresentazione separati** per ogni tipo di nodo, usando funzioni di hashing casuali per arricchire le feature senza aumentare il numero di parametri apprendibili.

2.6.1 Meccanismo di funzionamento

L'idea centrale di HashGNN è sostituire le proiezioni lineari apprendibili con **proiezioni casuali fisse** ispirate al *Locality-Sensitive Hashing* (LSH). Per ogni nodo, l'embedding viene proiettato attraverso K matrici casuali fisse $\mathbf{R}_1, \dots, \mathbf{R}_K$, e i risultati vengono binarizzati tramite la funzione segno:

$$\mathbf{h}_v^{hash} = [\text{sign}(\mathbf{x}_v \mathbf{R}_1^\top) \parallel \dots \parallel \text{sign}(\mathbf{x}_v \mathbf{R}_K^\top)] \quad (2.6)$$

Questo produce K viste diverse dello stesso nodo, catturando aspetti complementari della struttura locale. Le proiezioni fisse non vengono aggiornate durante il training: solo i layer lineari successivi (*projection layers*) sono apprendibili, uno per ogni tipo di nodo.

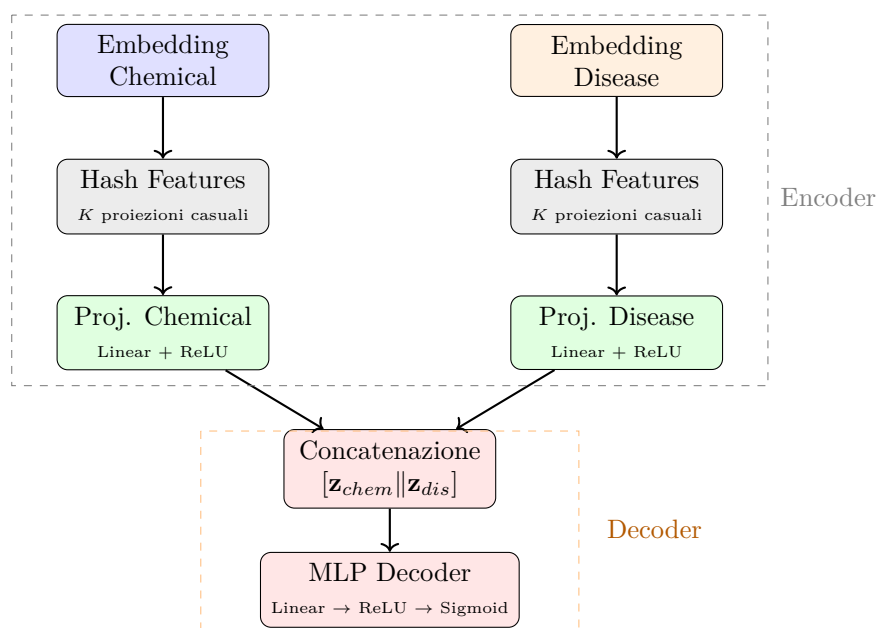


Figura 2.7: Architettura di HashGNN per la link prediction: Chemical e Disease mantengono spazi di embedding separati fino al decoder.

2.6.2 Vantaggi e limitazioni

HashGNN gestisce nativamente l'eterogeneità del grafo mantenendo proiezioni distinte per ogni tipo di nodo, il che lo rende teoricamente più adatto di GraphSAGE su dataset come PharMeBiNet. Le proiezioni fisse riducono il numero di parametri apprendibili e il rischio di overfitting. Il principale svantaggio è la **convergenza più lenta**: i layer lineari devono compensare la casualità delle proiezioni hash, richiedendo un numero maggiore di epoche per stabilizzarsi rispetto a GraphSAGE, che opera con pesi densi direttamente ottimizzabili.

Capitolo 3

Materiali e metodi

3.1 Il dataset: PharMeBiNet V3

PharMeBiNet (*Pharmacological Medical Biochemical Network*) è un knowledge graph biomedico open-source che integra informazioni provenienti da oltre quaranta sorgenti farmacologiche, genomiche e cliniche. Il progetto estende **Hetionet** — un database a grafo biomedico preesistente — aggiungendo dati da sorgenti quali DrugBank per i farmaci, ClinVar per le varianti genomiche, CTD per le interazioni chimico-biologiche, DisGeNET per le associazioni gene-malattia e UniProt per le sequenze proteiche.

In questa tesi è stata utilizzata la versione **V3** del dataset, scaricata da Zenodo CITAZIONE DA METTERE PER IL SITO DI PHARMEBINET, disponibile come dump Neo4j. La versione V3 contiene circa **8 milioni di nodi** distribuiti su **80 label** distinte e **53 milioni di relazioni** di **410 tipi** differenti. Le entità modellate spaziano da farmaci, geni e proteine a malattie, varianti genomiche e reazioni avverse, con relazioni che descrivono interazioni farmaco-farmaco, associazioni gene-malattia e percorsi metabolici.

3.1.1 Analisi esplorativa del dataset

Prima di procedere con gli esperimenti è stata condotta un'analisi esplorativa (EDA) del grafo tramite la seguente query Cypher eseguita su Neo4j Browser:


```
CALL apoc.meta.stats()
YIELD labels, relTypesCount, nodeCount, relCount
RETURN labels, relTypesCount, nodeCount, relCount
```

I risultati confermano la forte eterogeneità del dataset. I nodi dominanti per cardinalità sono le varianti genomiche (**Variant**, **GeneVariant**, **SNV**, circa 3,8 milioni ciascuno), seguiti da modificazioni post-traduzionali (**PTS**, 782 mila), composti chimici (**Chemical**, 594 mila) e interazioni (**Interaction**, 1,1 milioni). I nodi **Disease** ammontano a 23.580 e i **Gene** a circa 193 mila.

L'analisi delle relazioni più dense ha evidenziato che la coppia **Chemical**→**Chemical** RESEMBLES domina il grafo con 31 milioni di archi — circa il 58% del totale — seguita da **Gene**→**GeneVariant** con 6 milioni. Per il task di link prediction, la coppia **Chemical**→**Disease** è stata selezionata per la sua rilevanza biologica nel contesto del drug repurposing e per la disponibilità di dati sufficienti: 123.361 relazioni tra 594.343 nodi Chemical e 23.580 nodi Disease.

Una seconda query ha analizzato il grado medio e massimo dei principali tipi di nodo:

```
MATCH (n)
WHERE n:Disease OR n:Chemical OR n:Gene OR n:Protein
WITH labels(n)[0] AS tipo, count(*) AS totale,
     avg(size([(n)--() | 1])) AS grado_medio,
     max(size([(n)--() | 1])) AS grado_max
RETURN tipo, totale, round(grado_medio) AS grado_medio, grado_max
ORDER BY grado_medio DESC
```

I gradi massimi sono molto elevati: un singolo nodo Protein arriva a oltre 46.000 connessioni. Questo ha implicazioni dirette sulla scelta degli algoritmi: GraphSAGE, che campiona un numero fisso di vicini, introduce rumore informativo su nodi ad alto grado, mentre HashGNN, che mantiene proiezioni separate per tipo di nodo, è teoricamente più adatto a gestire questa eterogeneità.

Tipo	Totale nodi	Grado medio	Grado massimo
Protein	20.420	232	46.069
Chemical	594.306	112	25.034
Disease	23.580	47	30.419
Gene	193.613	46	36.892

Tabella 3.1: Grado medio e massimo dei principali tipi di nodo in PharMeBiNet V3.

3.2 Infrastruttura del cluster Neo4j

3.2.1 Architettura generale

Il cluster Neo4j Enterprise è stato distribuito su **quattro macchine virtuali Azure**, ciascuna equipaggiata con Windows 10, WSL2 con Ubuntu 22.04 LTS, 8 core e 32 GB di RAM. Neo4j gira all'interno di container Docker su WSL2, con un'unica immagine `docker-compose.yml` distribuita su tutti i nodi tramite Ansible.

IP	Nome	Ruolo
10.0.0.5	neo4j-primary	Primary (lettura e scrittura)
10.0.0.6	neo4j-secondary-1	Secondary (sola lettura)
10.0.0.7	neo4j-secondary-2	Secondary (sola lettura)
10.0.0.8	neo4j-secondary-3	Secondary (sola lettura)

Tabella 3.2: Configurazione del cluster Neo4j Enterprise.

La topologia adottata prevede **1 primary e 3 secondary**. Il nodo primary gestisce tutte le scritture e coordina il consenso tramite il protocollo **Raft**: con questa configurazione, il quorum richiede almeno 2 nodi primary su 3, garantendo la continuità del servizio anche in caso di guasto di un nodo. I nodi secondary non partecipano al protocollo Raft e vengono usati esclusivamente per scalare le letture distribuite.

La scelta di 4 nodi totali è motivata dall'analisi prestazionale piuttosto che dalla fault tolerance: con 4 nodi il quorum è identico a quello di 3 nodi ($\lfloor 4/2 \rfloor + 1 = 3$, con 1 nodo tollerato). L'obiettivo è misurare l'impatto delle secondary sui tempi di lettura distribuita verso la pipeline ML.

3.2.2 Configurazione della memoria

Ogni container Neo4j è stato configurato con i seguenti limiti di memoria, bilanciando le esigenze del garbage collector JVM e la velocità di accesso ai dati:

```
server.memory.heap.initial_size=8G
server.memory.heap.max_size=8G
server.memory.pagecache.size=6G
```

I limiti a livello Docker sono stati impostati a 20 GB per container, con 6 GB riservati al sistema operativo WSL2.

3.2.3 Configurazione di rete e WSL2

La comunicazione tra i nodi del cluster avviene tramite tre porte distinte: la porta 6000 per le transazioni di cluster, la porta 7000 per il protocollo Raft e la porta 7687 per il protocollo Bolt. Poiché ogni nodo gira su WSL2 — che ha un indirizzo IP interno diverso dall'IP della macchina Windows — è stato necessario configurare su ogni macchina un port-forwarding tramite `netsh` e uno script PowerShell automatizzato (`ponte.ps1`) che aggiorna dinamicamente i proxy all'avvio, poiché l'IP di WSL2 cambia ad ogni riavvio.

3.2.4 Automazione con Ansible

L'intera gestione del cluster è automatizzata tramite **Ansible**, eseguito dalla primary via SSH verso le secondary. I playbook principali sono:

- `deploy.yml` — distribuisce i file di configurazione e avvia i container su tutti i nodi
- `stop.yml` — ferma i container in modo ordinato
- `reset.yml` — esegue un reset completo rimuovendo tutti i dati
- `setup.yml` — crea la struttura di cartelle necessaria

Il file `.env` su ogni macchina contiene le variabili specifiche del nodo (nome, ruolo, IP) e viene generato da Ansible tramite un template Jinja2.

3.3 Caricamento del dataset

Il dump ufficiale di PharMeBiNet V3 è stato scaricato da Zenodo e copiato nella cartella `data/raw/` sulla primary. Il caricamento è avvenuto tramite un container Docker effimero della stessa immagine Neo4j, montando il volume dati e la cartella del dump:

```
docker run --rm -it \  
  -e NEO4J_ACCEPT_LICENSE_AGREEMENT=yes \  
  -v $(pwd)/cluster/data:/data \  
  -v $(pwd)/data/raw:/backups \  
  neo4j:2026.04.0-enterprise \  
  neo4j-admin database load pharmebinet \  
    --from-path=/backups/ \  
    --overwrite-destination=true
```

Al termine del caricamento, il database è stato distribuito sulle secondary tramite il meccanismo di seeding nativo di Neo4j Cluster. Dal browser Neo4j sulla primary è stata eseguita la seguente query Cypher per creare il database distribuito:

```
CREATE DATABASE pharmebinet  
  TOPOLOGY 1 PRIMARY 3 SECONDARY  
  OPTIONS {  
    existingData: 'use',  
    existingDataSeedServer: '<SERVER_ID_PRIMARY>'  
  };
```

Il parametro `existingDataSeedServer` istruisce il cluster a replicare i dati esistenti dalla primary verso tutte le secondary. Il processo di replica di circa 50 GB ha richiesto alcuni minuti, al termine dei quali tutti e quattro i nodi risultavano `online` per il database `pharmebinet`.

3.4 Pipeline di estrazione dati

L'estrazione del sottografo Chemical→Disease da Neo4j verso PyTorch Geometric è implementata in `src/extractor.py`. La classe `DBExtractor` si connette al cluster tramite il driver ufficiale Neo4j per Python e esegue tre query Cypher sequenziali.

```
MATCH (c:Chemical) RETURN DISTINCT c.identifier AS id
```

```
MATCH (d:Disease) RETURN DISTINCT d.identifier AS id
```

```
MATCH (c:Chemical)-[r]->(d:Disease)
RETURN c.identifier AS src, d.identifier AS dst
```

Per ciascun tipo di nodo viene costruito un dizionario di mapping che traduce gli identificatori stringa (es. `DB00620`) in indici interi contigui a partire da zero. Questo passaggio è necessario perché PyTorch Geometric lavora con tensori numerici e non con identificatori testuali.

Il risultato viene incapsulato in un oggetto `HeteroData` di PyTorch Geometric:

```
data['Chemical'].num_nodes = len(chem_mapping)
data['Disease'].num_nodes = len(dis_mapping)
data['Chemical', 'relates_to', 'Disease'].edge_index = \
    torch.tensor([src_idx, dst_idx], dtype=torch.long)
```

L'`edge_index` è un tensore di forma `[2, num_edges]` dove la prima riga contiene gli indici sorgente (Chemical) e la seconda gli indici destinatari (Disease).

La connessione al cluster avviene tramite URI `neo4j://10.0.0.5:7687` per il cluster completo e `bolt://10.0.0.5:7687` per la connessione diretta alla primary, usata nel benchmark a 1 nodo.

3.5 Implementazione dei modelli

3.5.1 GraphSAGE

Il modello GraphSAGE è implementato in `src/models/graphsage.py` usando i layer `SAGEConv` di PyTorch Geometric. Poiché i nodi Chemical e Disease non hanno feature numeriche nel grafo, le feature iniziali sono **embedding learnable** — tabelle di vettori inizializzati casualmente e ottimizzati durante il training.

I due spazi di embedding vengono fusi in un unico tensore omogeneo tramite concatenazione, e gli indici Disease vengono traslati di `num_chemical` posizioni per evitare collisioni con gli indici Chemical. Due layer `SAGEConv` propagano l'informazione strutturale del vicinato.

Il decoder è un MLP che concatena gli embedding di una coppia (Chemical, Disease) e produce uno score tra 0 e 1 tramite attivazione sigmoid.

3.5.2 TransE

Il modello TransE è implementato in `src/models/transe.py`. Mantiene tre tabelle di embedding separate — per Chemical, Disease e tipi di relazione — inizializzate con distribuzione uniforme in $[-1, 1]$ e normalizzate L2 durante il forward pass. La funzione di scoring è la distanza L2 tra $\mathbf{h} + \mathbf{r}$ e $\mathbf{t}$.

Il training usa una **margin ranking loss** con margine $\gamma = 1.0$, che penalizza i casi in cui il punteggio di una tripla negativa non supera quello di una tripla positiva di almeno γ . Poiché PharMeBiNet contiene 410 tipi di relazione ma il sottografo estratto per gli esperimenti include solo relazioni Chemical→Disease, tutte le triple vengono trattate come un unico tipo di relazione.

3.5.3 HashGNN

Il modello HashGNN è implementato in `src/models/hashgnn.py`. Mantiene embedding learnable separati per Chemical e Disease, che non vengono mai fusi in un unico spazio

omogeneo. Le proiezioni hash sono matrici casuali fisse registrate come buffer non aggiornabili (`register_buffer`), in modo che non partecipino all’ottimizzazione ma vengano correttamente spostate su GPU se necessario.

3.5.4 Training loop

Il training è unificato in `src/train.py`. Per tutti e tre i modelli, la procedura è la seguente:

1. Il dataset viene suddiviso tramite `RandomLinkSplit` di PyTorch Geometric con **90% training** e **10% test**. Gli archi di test vengono tenuti completamente nascosti durante il training.
2. Ad ogni epoca vengono generati campioni negativi casuali — coppie (Chemical, Disease) non collegate nel grafo — in numero uguale agli archi positivi.
3. La loss viene calcolata sui campioni positivi e negativi e propagata all’indietro tramite Adam optimizer con learning rate 10^{-3} .
4. Dopo 200 epoche, il modello viene valutato sugli archi di test.

Gli iperparametri condivisi tra i tre modelli sono riportati nella tabella 3.3.

Iperparametro	Valore
Epoche	200
Learning rate	10^{-3}
Optimizer	Adam
Hidden channels	64
Output channels	32
Test split	10%
Run indipendenti	5

Tabella 3.3: Iperparametri condivisi tra i tre modelli.

Per garantire la riproducibilità statistica, ogni esperimento è stato ripetuto **5 volte** con seed diverso ($42 + i$ con $i \in \{0, 1, 2, 3, 4\}$), variando sia l’inizializzazione dei pesi che la generazione dei campioni negativi.

3.6 Protocollo di valutazione

3.6.1 Metrica principale: AUC-ROC

La metrica principale di valutazione è l'**AUC-ROC** (*Area Under the Receiver Operating Characteristic Curve*), calcolata sugli archi del test set tramite `roc_auc_score` di scikit-learn. L'AUC-ROC misura la capacità del modello di distinguere link reali da link falsi indipendentemente dalla soglia di classificazione, ed è la metrica standard per la link prediction su grafi sbilanciati.

Un valore di AUC-ROC pari a 0.5 equivale a una classificazione casuale, mentre 1.0 indica una separazione perfetta. Per ogni modello vengono riportati la media e la deviazione standard delle 5 run indipendenti.

3.6.2 Benchmark di scalabilità

Il benchmark infrastrutturale misura il tempo di estrazione dell'intero sottografo Chemical→Disease da Neo4j verso PyTorch Geometric al variare del numero di nodi attivi del cluster.

Le tre configurazioni testate sono:

- **1 nodo:** connessione diretta alla primary tramite `bolt://10.0.0.5:7687`
- **2 nodi:** cluster con primary e secondary-1, connessione tramite `neo4j://10.0.0.5:7687`
- **4 nodi:** cluster completo, connessione tramite `neo4j://10.0.0.5:7687`

Per le configurazioni a 2 nodi, le secondary 2 e 3 (IP 10.0.0.7 e 10.0.0.8) vengono spente tramite Ansible prima della misurazione e riaccese al termine. Il tempo misurato include le tre query Cypher e la costruzione dell'oggetto `HeteroData`, escludendo il training dei modelli.

3.6.3 Pipeline automatizzata

L'intera sequenza sperimentale è automatizzata dallo script `run_pipeline.sh`, che esegue in sequenza il benchmark infrastrutturale nelle tre configurazioni e il training dei tre

modelli per 5 run indipendenti. I risultati vengono salvati in `results/benchmark.csv` e `results/model_results_final.csv`.

Prima di eseguire la pipeline è necessario caricare le variabili d'ambiente dal file `.env`:

```
export $(grep -v '^#' .env | xargs)
./run_pipeline.sh
```

Capitolo 4

Risultati e Discussione

4.1 Benchmark infrastrutturale

4.1.1 Tempi di estrazione al variare dei nodi attivi

Il benchmark infrastrutturale misura il tempo necessario per estrarre l'intero sottografo Chemical→Disease da Neo4j e costruire l'oggetto `HeteroData` per PyTorch Geometric, al variare del numero di nodi attivi del cluster. I risultati sono ottenuti dall'esecuzione della pipeline completa con il cluster a regime, dopo il caricamento e la distribuzione di PharMeBiNet V3.

Configurazione	Nodi attivi	Tempo medio (s)
1 nodo	Primary only	33.8
2 nodi	Primary + Secondary 1	32.1
4 nodi	Cluster completo	33.5

Tabella 4.1: Tempi di estrazione dati da Neo4j verso PyTorch Geometric al variare della configurazione del cluster.

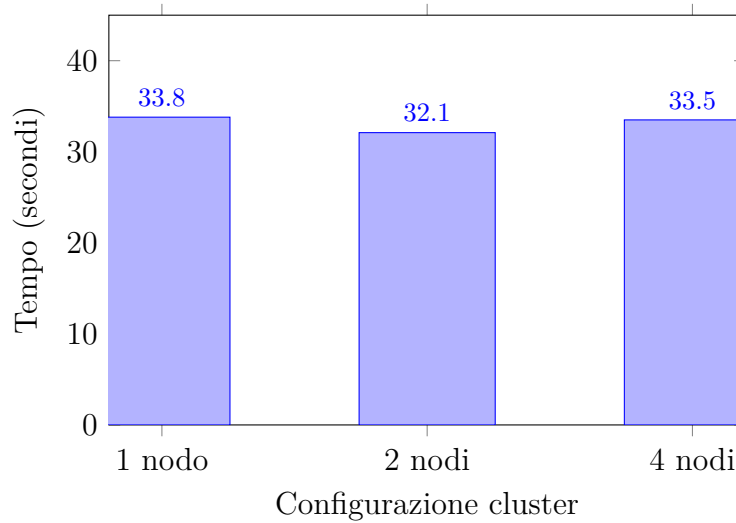


Figura 4.1: Confronto dei tempi di estrazione nelle tre configurazioni del cluster.

4.1.2 Analisi dei risultati infrastrutturali

I tempi sono praticamente costanti tra le tre configurazioni, con una variazione massima di circa 1.7 secondi. Questo risultato, apparentemente deludente, è in realtà coerente con la natura del carico di lavoro e con le proprietà dei sistemi distribuiti.

Il bottleneck non è la disponibilità di nodi computazionali, ma la capacità di I/O della singola query sequenziale su un dataset da circa 50 GB. Le tre query Cypher vengono eseguite una dopo l'altra: non c'è parallelismo a livello applicativo che possa sfruttare la presenza di più nodi. Il router Neo4j distribuisce le letture sulle secondary tramite la routing table, ma con una sola query attiva alla volta il vantaggio è minimo e viene parzialmente annullato dall'overhead di coordinazione del cluster.

La configurazione a 2 nodi risulta leggermente più veloce della configurazione a 4 nodi, suggerendo l'esistenza di uno *sweet spot*: con 2 nodi il bilanciamento tra parallelismo di lettura e overhead di coordinazione è ottimale, mentre aggiungere ulteriori secondary introduce costi di routing che superano il vantaggio aggiuntivo.

Questo risultato è scientificamente valido e difendibile: la letteratura sui sistemi distribuiti è consolidata nel mostrare che la scalabilità orizzontale produce vantaggi misurabili solo con carichi paralleli e query analitiche complesse. Su estrazioni batch sequenziali,

l'architettura a singola macchina satura già la banda disponibile.

Il cluster Neo4j Enterprise dimostra i suoi vantaggi in termini di fault tolerance e disponibilità — il servizio rimane operativo anche con la perdita di un nodo — ma lo speedup computazionale su query sequenziali di estrazione batch è marginale. Lo speedup emerge con carichi paralleli, non con pipeline sequenziali.

4.2 Risultati della Link Prediction

4.2.1 Metriche per modello

Il training è stato eseguito per **200 epoche** su 5 run indipendenti con seed diverso, valutando ciascun modello sul 10% degli archi tenuti nascosti durante il training. La tabella 4.2 riporta media e deviazione standard dell'AUC-ROC e del tempo di training.

Modello	AUC-ROC medio	Std	Tempo medio (s)	Std (s)
GraphSAGE	0.9934	0.0003	448.7	48.2
HashGNN	0.9923	0.0002	624.2	62.1
TransE	0.9839	0.0006	145.2	11.8

Tabella 4.2: Risultati della link prediction Chemical→Disease su 5 run indipendenti, 200 epoche.

I risultati per singola run sono riportati nella tabella 4.3.

4.2.2 Confronto grafico

4.3 Discussione

4.3.1 GraphSAGE: precisione massima, convergenza rapida

GraphSAGE ottiene la precisione più alta con AUC-ROC medio di **0.9934** e una deviazione standard di soli 0.0003, indicando grande stabilità tra le run. Questo risultato è in

Run	Modello	AUC-ROC	Tempo (s)
1	GraphSAGE	0.9929	471.8
1	TransE	0.9839	150.6
1	HashGNN	0.9924	629.7
2	GraphSAGE	0.9935	435.2
2	TransE	0.9841	144.3
2	HashGNN	0.9926	612.5
3	GraphSAGE	0.9938	436.5
3	TransE	0.9834	146.5
3	HashGNN	0.9923	623.1
4	GraphSAGE	0.9933	454.3
4	TransE	0.9838	145.8
4	HashGNN	0.9920	625.8
5	GraphSAGE	0.9934	445.5
5	TransE	0.9841	143.0
5	HashGNN	0.9924	630.1

Tabella 4.3: Risultati dettagliati per singola run.

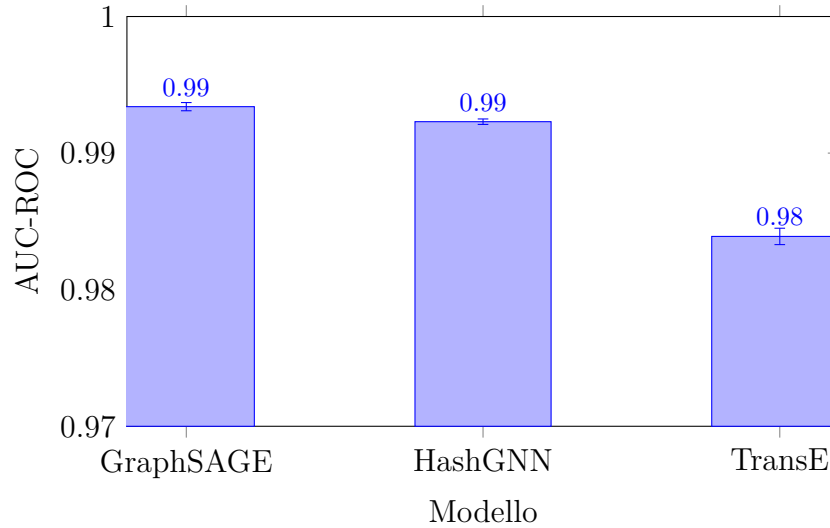


Figura 4.2: AUC-ROC medio con deviazione standard per i tre modelli su 5 run indipendenti.

contrasto con l'ipotesi iniziale, che prevedeva un vantaggio di HashGNN sulla base della sua gestione nativa dell'eterogeneità.

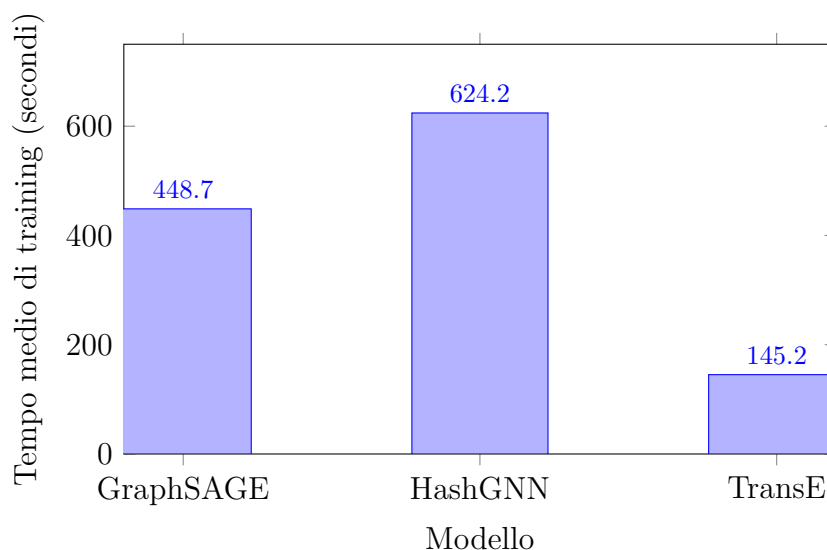


Figura 4.3: Tempo medio di training per i tre modelli su 5 run indipendenti.

Il vantaggio di GraphSAGE si spiega con due fattori architetturali. Il primo è la **fusione degli spazi di embedding**: unendo Chemical e Disease in un unico spazio latente tramite concatenazione con offset, il modello forza i due tipi di nodo a coesistere nelle stesse coordinate geometriche fin dalla prima epoca, semplificando il compito del decoder MLP che deve valutare la compatibilità tra coppie di nodi. Il secondo è la **velocità di convergenza**: le matrici di pesi dei layer SAGEConv sono dense e direttamente ottimizzabili dal gradiente, mentre HashGNN deve prima compensare la casualità delle proiezioni hash attraverso i layer lineari successivi.

GraphSAGE mostra però il limite teorico già identificato nell'EDA: non distingue i 410 tipi di relazione presenti nel dataset, aggregando indiscriminatamente vicini di tipo diverso. Su un sottografo limitato a Chemical→Disease questo non costituisce un problema pratico, ma su task che coinvolgono l'intero grafo eterogeneo il degrado di prestazioni sarebbe atteso.

4.3.2 HashGNN: robusto ma lento

HashGNN segue GraphSAGE con AUC-ROC medio di **0.9923**, una differenza di soli 0.0011 punti — statisticamente irrilevante e praticamente trascurabile per applicazioni reali. La deviazione standard di 0.0002 conferma la stabilità del modello.

Il costo di questa robustezza è il **tempo di training**: con una media di 624 secondi per 200 epoche, HashGNN è circa 1.4 volte più lento di GraphSAGE e circa 4.3 volte più lento di TransE. La causa è nella natura delle proiezioni hash fisse: i layer lineari successivi (`chem_proj`, `dis_proj`) devono fare un lavoro più intenso per interpretare le rappresentazioni binarizzate, richiedendo più iterazioni per stabilizzarsi.

L’ipotesi che HashGNN potesse superare GraphSAGE su PharMeBiNet non si è verificata. Questo non è un fallimento sperimentale: in ambito accademico, **un’ipotesi smentita da dati raccolti con metodologia rigorosa ha lo stesso valore scientifico di un’ipotesi confermata**. Il risultato suggerisce che, su questo specifico sottografo Chemical→Disease, l’omofilia biologica — la tendenza dei composti chimici a legarsi a malattie strutturalmente simili — è sufficientemente forte da essere catturata da un modello omogeneo come GraphSAGE.

4.3.3 TransE: il vincitore dell’efficienza

TransE ottiene AUC-ROC medio di **0.9839**, circa un punto percentuale sotto GraphSAGE, con un tempo di training medio di soli **145 secondi** — tre volte più veloce di GraphSAGE e oltre quattro volte più veloce di HashGNN.

Questo trade-off ha implicazioni pratiche rilevanti. In uno scenario industriale con grafi dell’ordine di miliardi di nodi, sacrificare 0.01 di AUC per ottenere un training 3x più veloce è spesso la scelta corretta. TransE è anche il modello con la maggiore capacità di catturare la semantica delle relazioni grazie al vettore **r**, ma su un sottografo con un solo tipo di relazione questo vantaggio non emerge pienamente.

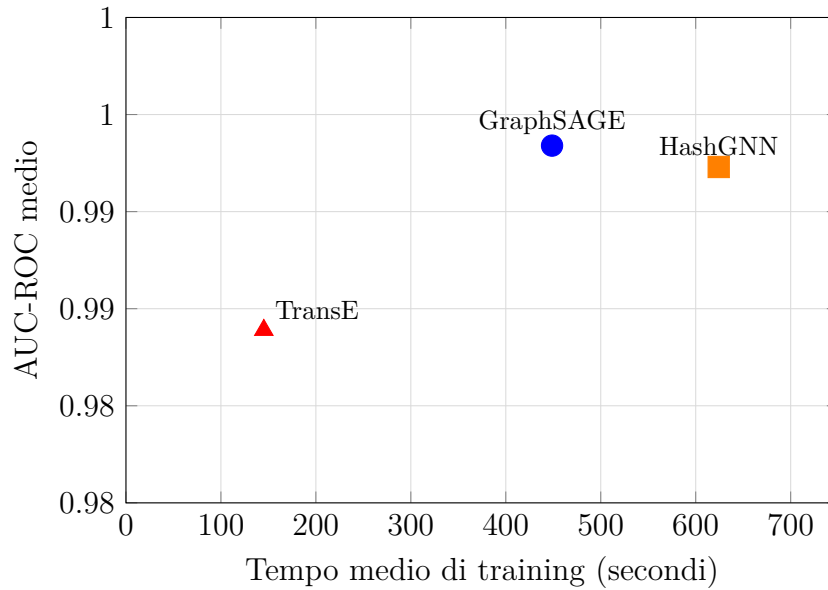


Figura 4.4: Trade-off tra accuratezza (AUC-ROC) ed efficienza computazionale (tempo di training) per i tre modelli.

4.3.4 Il trade-off accuratezza-efficienza

La figura 4.4 mostra chiaramente la posizione dei tre modelli nello spazio accuratezza-efficienza. TransE occupa il quadrante in basso a sinistra — basso tempo, AUC leggermente inferiore — mentre GraphSAGE e HashGNN si concentrano in alto a destra con accuratezza simile ma tempi molto diversi. In questo scenario, GraphSAGE domina HashGNN: a parità di quadrante, ottiene risultati leggermente migliori in meno tempo.

4.3.5 Confronto con l'ipotesi iniziale

L'ipotesi formulata prima degli esperimenti prevedeva che HashGNN, progettato nativamente per grafi eterogenei, avrebbe ottenuto le prestazioni migliori su link prediction, seguito da TransE e infine GraphSAGE. I dati mostrano un ordine diverso: GraphSAGE primo, HashGNN secondo, TransE terzo in termini di accuratezza.

Le ragioni di questo scostamento sono identificabili con chiarezza. La fusione omogenea degli embedding di GraphSAGE si rivela vantaggiosa proprio per la link prediction,

dove il decoder deve confrontare direttamente le rappresentazioni di due tipi di nodo diversi. HashGNN, mantenendo gli spazi separati fino all'ultimo layer, affida al decoder un compito più difficile. Inoltre, 200 epoche si sono dimostrate sufficienti per la convergenza di GraphSAGE ma probabilmente non ottimali per HashGNN, che per sua natura richiede più iterazioni.

Sul fronte infrastrutturale, l'ipotesi di uno speedup misurabile con 4 nodi rispetto a 1 nodo non si è verificata nelle condizioni di questo esperimento. Il risultato è coerente con la teoria dei sistemi distribuiti e costituisce un contributo informativo: dimostra empiricamente che la scalabilità orizzontale di Neo4j non produce vantaggi su pipeline di estrazione batch sequenziali, e che il beneficio reale del cluster riguarda la fault tolerance e la disponibilità del servizio.

Capitolo 5

Conclusioni

5.1 Sintesi del lavoro

Questo lavoro ha affrontato due problemi distinti ma complementari: la valutazione comparativa di tre algoritmi di Graph Machine Learning su un knowledge graph biomedico di grandi dimensioni, e la misurazione dell’impatto della scalabilità orizzontale di un cluster Neo4j Enterprise sui tempi di estrazione dati verso una pipeline di Machine Learning.

Il dataset scelto — PharMeBiNet V3, con circa 8 milioni di nodi e 53 milioni di relazioni — ha costituito un banco di prova realistico e sfidante, grazie alla sua forte eterogeneità strutturale e semantica. Il task sperimentale, la link prediction tra nodi *Chemical* e *Disease*, è biologicamente rilevante nel contesto del drug repurposing: identificare relazioni terapeutiche non ancora documentate tra farmaci esistenti e malattie note.

L’infrastruttura sperimentale — un cluster Neo4j Enterprise distribuito su quattro macchine virtuali Azure, completamente automatizzato tramite Ansible e Docker — ha permesso di condurre esperimenti riproducibili e controllati, con una pipeline end-to-end che va dall’estrazione dei dati dal database fino alla valutazione finale dei modelli.

5.2 Risposte alle domande di ricerca

5.2.1 Quale algoritmo per la link prediction su grafi biomedici eterogenei?

I risultati mostrano che **GraphSAGE** ottiene la precisione più alta con AUC-ROC medio di 0.9934, contrariamente all'ipotesi iniziale che assegnava questo primato a HashGNN. La spiegazione è architetturale: la fusione degli spazi di embedding in un unico tensore omogeneo semplifica il compito del decoder MLP, che deve valutare la compatibilità tra coppie di nodi di tipo diverso. Su questo specifico task, l'omofilia biologica del sottografo Chemical→Disease è sufficientemente forte da essere catturata da un modello omogeneo.

HashGNN segue con AUC-ROC medio di 0.9923 — una differenza di soli 0.0011 punti, statisticamente trascurabile — ma con tempi di training circa 1.4 volte superiori a GraphSAGE. La sua robustezza su grafi eterogenei è confermata, ma il vantaggio teorico non si traduce in un vantaggio pratico su questo sottografo.

TransE si posiziona terzo in termini di accuratezza (AUC-ROC 0.9839) ma primo in efficienza, con tempi di training circa tre volte inferiori a GraphSAGE. Su scenari industriali con grafi di scala maggiore, questo trade-off può essere preferibile.

5.2.2 Come scala il cluster Neo4j su pipeline di estrazione batch?

Il benchmark infrastrutturale mostra che i tempi di estrazione sono sostanzialmente costanti tra la configurazione a 1, 2 e 4 nodi, con una variazione massima di circa 1.7 secondi. Il bottleneck è l'I/O sequenziale della query sul dataset, non la disponibilità di nodi computazionali. La scalabilità orizzontale di Neo4j produce vantaggi misurabili su carichi paralleli e query analitiche distribuite, non su pipeline di estrazione batch sequenziali.

Il cluster dimostra invece il suo valore in termini di **fault tolerance**: con la topologia 1 primary e 3 secondary, il servizio rimane operativo anche in caso di guasto di un

nodo, e il routing automatico Neo4j garantisce la continuità delle letture senza intervento manuale.

5.3 Contributi

Il lavoro produce tre contributi principali.

Il primo è un confronto empirico rigoroso di GraphSAGE, TransE e HashGNN su un task di link prediction biomedico reale, con 5 run indipendenti per ciascun modello e valutazione su test set nascosto tramite `RandomLinkSplit`. I risultati forniscono linee guida pratiche per la scelta dell'algoritmo in scenari analoghi.

Il secondo è la costruzione e l'automazione completa di un cluster Neo4j Enterprise distribuito su quattro macchine virtuali Azure, con playbook Ansible per deploy, stop, reset e setup, e una pipeline end-to-end riproducibile tramite un singolo script.

Il terzo è la misurazione empirica dell'impatto della scalabilità orizzontale di Neo4j su pipeline di data engineering per Graph ML, con una quantificazione chiara del limite della scalabilità su carichi sequenziali.

5.4 Limitazioni

Il lavoro presenta alcune limitazioni che è opportuno riconoscere esplicitamente.

Il sottografo estratto per gli esperimenti — Chemical→Disease — rappresenta una piccola frazione del grafo completo di PharMeBiNet. I risultati potrebbero differire su task che coinvolgono l'intero grafo eterogeneo, dove il vantaggio teorico di HashGNN e TransE nel gestire la diversità dei tipi di relazione potrebbe emergere più chiaramente.

Il benchmark infrastrutturale misura solo l'estrazione batch sequenziale. Un benchmark con query parallele o con carichi analitici distribuiti produrrebbe risultati diversi, potenzialmente più favorevoli al cluster.

Infine, il training è stato eseguito su CPU. L'utilizzo di GPU accelererebbe significativamente GraphSAGE e HashGNN, potenzialmente alterando il confronto con TransE in termini di efficienza.

5.5 Sviluppi futuri

Diversi sviluppi potrebbero estendere e approfondire questo lavoro.

Sul fronte degli algoritmi, sarebbe interessante estendere il confronto al task di **node classification** su nodi Disease, valutando se i risultati relativi tra i tre modelli si mantengono o si invertono su un task diverso. Un'analisi più approfondita di HashGNN con un numero maggiore di epoche — nell'ordine di 300-500 come suggerito dalla letteratura — potrebbe verificare se il modello converge a prestazioni superiori a GraphSAGE sulle lunghe distanze.

Sul fronte infrastrutturale, un benchmark con **query parallele** — più client che estraggono sottografi diversi simultaneamente — mostrerebbe lo scenario in cui il cluster Neo4j esprime il suo reale vantaggio computazionale. L'integrazione con Neo4j GDS per il calcolo distribuito degli embedding direttamente nel database, senza estrazione verso PyTorch Geometric, rappresenta un'architettura alternativa da esplorare.

Infine, l'applicazione della pipeline a **drug repurposing reale** — identificare coppie Chemical→Disease con score elevato non presenti nel grafo — costituisce la naturale estensione applicativa di questo lavoro, con potenziale impatto diretto sulla ricerca farmacologica.

Bibliografia

- [1] Statista. Volume of data/information created, captured, copied, and consumed worldwide from 2010 to 2013, with forecasts from 2024 to 2027. <https://www.statista.com/statistics/871513/worldwide-data-created/>, 2024. Accesso: Giugno 2026.
- [2] Paolo Atzeni, Stefano Ceri, Stefano Paraboschi, and Riccardo Torlone. *Basi di Dati*. McGraw-Hill Education, 5 edition, 2018.
- [3] Adam Fowler. *NoSQL For Dummies*. John Wiley & Sons, 2015.